



Pattern Recognition

N°	Ce que tu vois dans le code	Ta réponse orale immédiate en entretien 🗣️
1	<code>res = async_fn()</code> (sans <code>await</code>)	"Je vois <code>async_fn()</code> , il manque <code>await</code> ! Sinon on manipule un objet <code><coroutine></code> non exécuté au lieu de la donnée réelle."
2	<code>asyncio.run(...)</code> dans FastAPI	"Pas de <code>asyncio.run()</code> dans FastAPI ! Nous sommes déjà dans l'Event Loop active d'Uvicorn, cela va lever une <code>RuntimeError</code> ."
3	<code>def fn(items=[])</code>	"Jamais <code>items=[]</code> en valeur par défaut ! C'est un argument mutable partagé entre toutes les requêtes utilisateurs (effet de bord global)."
4	<code>f = open(...)</code> (sans <code>with</code>)	"Toujours utiliser <code>with</code> / <code>async with</code> ! Cela garantit la fermeture déterministe des fichiers et sockets même en cas d'erreur (<code>Too many open files</code>)."
5	<code>except Exception: pass</code>	"Bannir <code>except: pass</code> ! Cela ravale l'exception en silence, détruit la chaîne d'observabilité (logs/Sentry) et crée un comportement zombie."

🗣️ Fiche Mémo : AI & Chatbots

N°	Ce que tu vois dans le code	Ta réponse orale immédiate en entretien 🗣️
1	<code>messages = messages[-10:]</code>	"Attention au slicing de mémoire ! Il faut toujours préserver l'index 0 (<code>messages[0]</code> = System Prompt) pour ne pas perdre les guardrails et consignes de l'agent."
2	<code>vector_store.search(query)</code> (sans filtre)	"Isolation Multi-Tenant obligatoire ! Il faut systématiquement injecter <code>filter={'tenant_id': ...}</code> dans le Vector Store pour éviter la fuite de données inter-clients (<i>Cross-tenant leakage</i>)."
3	<code>tool(**json.loads(args))</code>	"Validation Pydantic du Function Calling ! Ne jamais exécuter un JSON LLM brut sans schéma de validation (<code>model_validate_json()</code>) pour parer aux hallucinations et injections."
4	<code>mistral-large</code> partout	"Routage de Modèles (Model Tiering) ! Utiliser <code>mistral-small</code> pour les tâches simples (routage, extraction) afin d'optimiser les coûts par 10 et réduire la latence."

🗣️ Fiche Mémo : Evals, Tracing & Guardrails

N°	Ce que tu vois dans le code	Ta réponse orale immédiate en entretien 🧠
1	LLM-as-a-Judge dans <code>async def endpoint</code>	" Garder le LLM-as-a-Judge hors du chemin critique ! Préférer des Guardrails déterministes légers (<1ms) en synchrone et déporter le Judge en tâche d'arrière-plan (<code>BackgroundTasks</code>)."
2	Benchmark d'Evals avec <code>temp=0.7</code>	" Reproductibilité des Evals ! Forcer <code>temperature=0.0</code> et fixer un <code>random_seed=42</code> pour garantir des tests déterministes et valider les non-régressions en CI/CD."
3	Logs d'agent sans ID (<code>logger.info(...)</code>)	" Propager le <code>trace_id</code> ! Tout log ou sous-étape (<i>span</i>) d'un agent doit partager un <code>trace_id</code> unique pour pouvoir reconstruire l'arbre d'exécution APM."
4	Évaluations avec note de 1 à 5	" Critères Binaires Atomiques (Pass/Fail) ! Bannir les échelles de Likert 1-5 floues au profit de vérifications booléennes typées Pydantic (ex: <code>is_factual: bool</code>)."

N°	Ce que tu vois dans le code	Ta réponse orale immédiate en entretien 🧠
1	<code>return {"error": "Failed"}</code> dans un endpoint	" Lever <code>HTTPException</code> avec les bons codes ! Ne jamais renvoyer HTTP 200 OK avec du JSON d'erreur. Renvoyer du 400 (Client), 401 (Auth), 404 (Introuvable) ou 500 (Serveur)."
2	Retry HTTP sur un <code>POST</code> sans clé	" Contrôler l'Idempotence sur les Retries ! Injecter un en-tête <code>Idempotency-Key</code> (UUID) sur les requêtes de modification <code>POST</code> / <code>PUT</code> pour parer aux doublons et au <i>Thundering Herd</i> ."
3	Requete BDD dans un <code>for loop</code>	" Éliminer les requêtes N+1 ! Remplacer les requêtes unitaires en boucle par des requêtes groupées de batching (<code>IN (...)</code> en SQL) ou du <code>asyncio.gather()</code> ."
4	OCR / Traitement lourd de 30s synchrone	" Séparer Fast et Slow Paths ! Renvoyer immédiatement <code>HTTP 202 Accepted</code> avec un <code>job_id</code> pour les tâches lourdes et déléguer le traitement en Background Job."

N°	Ce que tu vois dans le code	Ta réponse orale immédiate en entretien 🧠
1	Inférence PyTorch sans décorateur	" Activer <code>@torch.no_grad()</code> et <code>model.eval()</code> ! " Obligatoire avant toute inférence pour éviter l'explosion VRAM (OOM) et désactiver le Dropout.
2	<code>torch.cat()</code> dans la boucle de tokens	" Pré-allouer le KV Cache ! " Bannir <code>torch.cat</code> dans la boucle autorégressive pour éviter les allers-retours $O(N^2)$ et maintenir un haut débit (tokens/s).
3	Multiplications matricielles <code>model</code> vs <code>inputs</code>	" Vérifier l'alignement des Devices CUDA/CPU ! " Toujours faire <code>tensor.to(device)</code> avant toute opération pour éviter les <code>RuntimeError</code> .
4	Cosine Similarity sans sécurité	" Sécuriser le Cosine Similarity avec ϵ !" Utiliser <code>torch.clamp(..., min=1e-8)</code> pour éliminer les divisions par zéro et la propagation de <code>NaN</code> .

5. Cache absent sur des appels LLM redondants

Problème : Rappeler le LLM avec le même prompt/contexte à chaque fois (ex: classification d'intent répétitive) sans cache.

Impact : Coût API inutile, latence inutile.

Correction : Cache simple (dict en mémoire, ou Redis en prod) sur les inputs répétitifs.

Fiche Mémo Supplémentaire : Bonnes Pratiques & Clean Code

N°	Ce que tu vois dans le code	Ta réponse orale immédiate en entretien 
2	Class qui fait tout (<code>GodClass</code> avec helpers statiques internes)	"Respecter le Principe de Responsabilité Unique (SRP) ! Extraire les fonctions utilitaires pures (lecture de fichiers, chunking) hors de la classe de service."
3	<code>json.loads(content)</code> suivi de <code>parsed["data"]["text"]</code>	"Sécuriser le Parsing JSON avec Pydantic ! Définir un modèle Pydantic (<code>ParsedText</code>) pour valider la structure du fichier et éviter les <code>KeyError</code> sauvages."
4	<code>logger.exception("..." + var)</code> avec f-string	"Utiliser le formatage paresseux dans les logs ! Préférer <code>logger.exception('... %s', var)</code> au lieu de f-strings pour préserver la stack trace complète dans Sentry/Datadog sans surcoût CPU."
5	Re-lever une exception capturée	"Conserver la Trace d'Erreur avec <code>raise</code> ! Dans un bloc <code>except</code> , toujours logger l'erreur avec <code>logger.exception</code> et utiliser <code>raise</code> pour ne pas couper la chaîne d'observabilité."
6	Endpoint FastAPI avec <code>request:</code> <code>Request</code> et <code>await request.json()</code>	"Utiliser des DTOs Pydantic pour les Endpoints ! Définir une classe <code>IngestPayload(BaseModel)</code> pour bénéficier du typage automatique, de la validation HTTP 422 et de la doc Swagger."

Fiche Mémo Supplémentaire : Agentic AI & Systems Architecture

N°	Ce que tu vois dans le code	Ta réponse orale immédiate en entretien 
1	<code>self.chat_history</code> stocké dans l'attribut <code>__init__</code> de l'agent	"Garantir des Agents Stateless en Production ! L'historique de conversation ne doit pas résider dans l'instance de l'agent (fuite mémoire et mélange inter-utilisateurs). Il doit être transmis par requête (<code>chat_history</code> dans la payload DTO)."
2	Accès direct <code>tool_call = message_obj.tool_calls[0]</code>	"Vérification Défensive des Tool Calls ! Toujours vérifier <code>if message_obj.tool_calls:</code> avant de sous-titrer <code>[0]</code> , sinon les requêtes conversationnelles classiques lèvent un <code>TypeError / IndexError</code> ."
3	Instanciation directe de services dans <code>__init__</code>	"Préférer l'Injection de Dépendances ! Découpler les services (<code>RagKnowledgeService</code> , <code>ToolExecutionService</code>) et les injecter dans le constructeur de l'agent pour faciliter les tests unitaires et le mocking."
4	Recherche RAG sans gestion d'erreur	"Implémenter un Graceful Fallback RAG ! Entourer la recherche vectorielle d'un <code>try/except</code> qui renvoie une liste vide <code>return []</code> en cas de panne de la Vector DB, permettant à l'agent de continuer à répondre."
5	Nom de modèle LLM écrit en dur dans <code>complete()</code>	"Paramétrer le Nom du Modèle ! Passer <code>model_name:</code> <code>str</code> en paramètre du service pour permettre le Model

N°	Ce que tu vois dans le code	Ta réponse orale immédiate en entretien 
		Tiering dynamique (ex: passer de <code>mistral-small</code> à <code>mistral-large</code> selon le besoin)."

Fiche Mémo Supplémentaire : Evals & Observabilité

N°	Ce que tu vois dans le code	Ta réponse orale immédiate en entretien 
1	Division directe <code>precision = TP / total_predictions</code>	"Protection contre la Division par Zéro ! Toujours vérifier <code>if total_predictions == 0: return 0.0</code> pour éviter qu'un dataset d'évaluations vide ou filtré ne fasse crasher le robot CI/CD."
2	Extraction du texte brut de réponse LLM-as-a-Judge	"Parsing Structuré Pydantic de la Sortie du Juge ! Utiliser <code>Model.model_validate_json()</code> sur la réponse du juge au lieu d'extraire une chaîne brute, garantissant des vérifications booléennes typées."
3	Boucle de Benchmark sans gestion d'erreur	"Isolation des Erreurs dans les Boucles d'Évaluation ! Encapsuler chaque itération du benchmark d'éval dans un <code>try/except</code> avec un <code>continue</code> pour qu'une panne réseau isolée n'annule pas les 99 autres tests."
4	Génération de <code>trace_id</code> dispersée	"Génération d'UUID à la Frontière HTTP ! Générer <code>trace_id = str(uuid.uuid4())</code> dès la première ligne de l'endpoint HTTP et le propager à toutes les sous-étapes pour garantir la traçabilité complète."

Fiche Mémo Supplémentaire : System Design & Async APIs

N°	Ce que tu vois dans le code	Ta réponse orale immédiate en entretien 
1	<code>time.sleep(1)</code> fixe dans une boucle de retry	"Implémenter un Exponential Backoff Async ! Remplacer le sleep fixe par <code>await asyncio.sleep(2 ** attempt)</code> pour étaler la charge de retry et éviter le <i>Thundering Herd</i> sur le serveur distant."
2	<code>requests.post()</code> synchrone dans un handler async	"Préférer <code>httpx.AsyncClient()</code> asynchrone ! Toujours utiliser <code>async with httpx.AsyncClient() as client:</code> pour ne pas bloquer l'Event Loop Uvicorn pendant les E/S réseau."
3	Construction dynamique de requêtes SQL <code>IN (...)</code>	"Sécuriser la construction SQL avec des Placeholders ! Générer <code>placeholders = ', '.join(['%s'] * len(ids))</code> et passer <code>tuple(ids)</code> en paramètre pour parer aux injections SQL."
4	URL de service externe écrite en dur dans le code	"Externaliser les URLs d'APIs tierces ! Charger les endpoints distants via des variables d'environnement (<code>os.environ['DB_LINK']</code>) pour autoriser le changement d'environnement (Dev/Staging/Prod)."
5	Code HTTP lors d'un échec de service tiers	"Utiliser le code HTTP 502 Bad Gateway ! Quand une API distante (indexeur, BDD) échoue après retries, lever <code>HTTPException(status_code=502)</code> pour indiquer qu'un service en amont est tombé."